



빌드시스템 / Gradle

Java 컴파일 & 실행

- 패키지가 아닌 경우
 - javac MyApplication.java
 - 소스파일이 있는 곳에 class 파일 생성
 - java MyApplication
- 패키지로 구성되어 있는 경우
 - 예) myproject/src/main/java/com/example/MyApplication.java
 - cd myproject
 - javac src/main/java/com/example/MyApplication.java
 - 소스파일이 있는 곳에 class 파일 생성
 - java -cp ./src/main/java com.example.MyApplication
- 소스 코드와 class 파일이 구분이 안됨, -d 옵션 사용해 해결가능
 - javac -d build src/main/java/com/example/MyApplication.java
 - myproject/build/com/example/MyApplication.class 파일 생성
 - java -cp ./build com.example.MyApplication
- 다양한 라이브러리의 증가 및 의존성 있는 라이브러리 관리가 매우 어려움
- 배포 버전을 만들기 매우 어려움
- 개발자간의 버전 관리가 어려움

```
package com.example;
public class MyApplication {
    public static void main(String[] args)
    {
        System.out.println("Hello");
    }
}
```



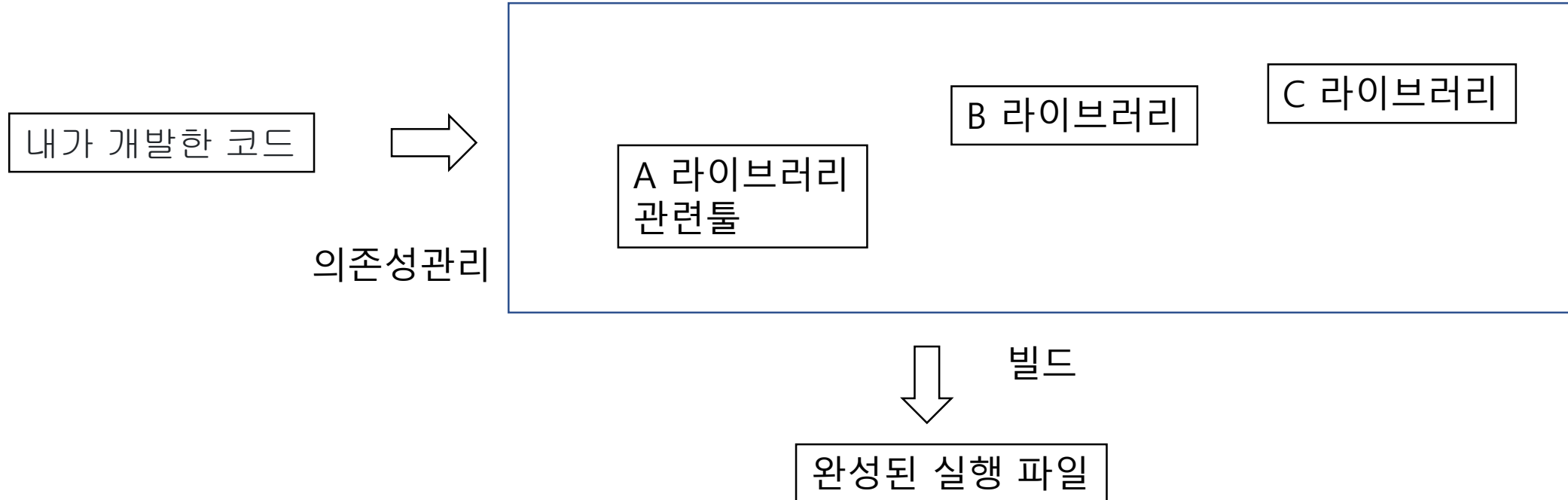
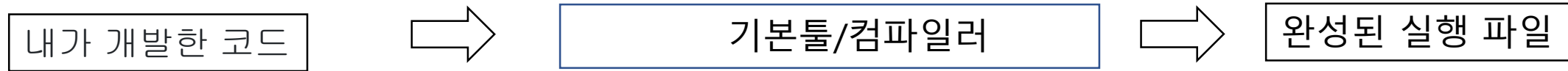
- gradle 다운로드
 - <https://gradle.org/install/>
- path 설정
 - set PATH= gradle bin폴더;%PATH%
- cd myapp
- build.gradle 작성
- gradle task 수행
- gradle compileJava
 - build/classes 폴더에 패키지 생성
- gradle run
 - build/classes 폴더에서 메인 클래스 실행
- gradle build
 - build/libs/__.jar 생성
- java -jar build/libs/myapp.jar

```
plugins {  
    id "application" // Java 애플리케이션 실행을 쉽게 해주는 플러그인  
                        // run, installDist, distZip  
}  
  
application {  
    mainClass = "com.example.MyApplication" // 프로그램 시작점(main 메서드 있는 클래스) 지정  
}  
  
jar {  
    manifest {  
        attributes "Main-Class": application.mainClass  
    }  
} // JAR 파일에 실행 정보 추가
```

빌드란



- 모든 빌드 시스템의 기본 목적은 엔지니어들이 작성한 소스 코드를 기계가 읽을 수 있는 바이너리로 변환하는 것
- 소스 코드 파일을 컴퓨터에서 실행할 수 있는 독립 소프트웨어 가공물로 변환하는 과정
- 소스코드(java), 프로젝트에서 쓰인 각각의 파일 및 자원등(.xml,jpa,jpg,properties)을 jvm이나 톰캣 같은 WAS가 인식할 수 있는 패키징하는 과정 및 결과물





- 어플리케이션 생성과정 중 빌드 관리 도구는 사용자가 직접 관리할 필요 없이 필요한 라이브러리들을 자동으로 관리
- 초기의 Java빌드 도구로는 Apache Ant를 많이 사용
- Ant는 스크립트 작성도 많고, 라이브러리 의존 관리가 되지 않음
- 빌드 관리 도구 기능
 - 종속성 다운로드
 - 소스코드를 바이너리코드로 컴파일
 - 바이너리 코드를 패키징
 - 테스트 실행
 - 프로덕션 시스템에 배포



- 정형화된 구조와 절차 제공
 - 컴파일(compile), 패키징(package), 테스트(test) 등의 작업 프로세스를 정형화
 - 표준에 따른 개발 가능
 - 프로젝트의 소스 트리 구성도 src/main/java 등과 같은 정형화된 패턴을 따르도록 가이드
- 디펜던시 라이브러리 해결을 자동화
 - 오픈소스 라이브러리의 경우 메이븐 중앙 저장소(central repository)에 모두 저장
 - 디펜던시가 있는 라이브러리의 명세만으로, 자동으로 다운로드하여 클래스 패스로 연결
- 라이브러리 관리 용이
 - 일반적으로 프로젝트 진행 시 사용되는 라이브러리를 형상관리(SCM : Software Configuration Management)도구에서 관리함(불필요한 공간 낭비, 버전별로 관리하는것도 매우 어려움)
 - 메이븐에서 라이브러리 전용 저장소(repository)를 제공하고 통합 관리 가능
- 라이프 사이클
 - 라이프 사이클에 따라서 연속적으로 단계를 실행
- 다양한 플러그인 활용
 - 플러그인을 사용하면 IDE 프로젝트로 자동 변환, 데이터베이스 통합, 다양한 리포트 생성등의 작업을 수행



- 자바 전용 빌드 프로젝트
 - Apache Ant 불편함을 해결하고자 만들어졌음
 - 정해진 Lifecycle에 의하여 작업을 수행하며, 전반적인 프로젝트 관리 기능 포함
 - 메이븐 이전에는 각 개발환경에 대한 빌드 프로세스를 파악하는데 시간이 걸림
 - Maven은 필요한 라이브러리를 pom.xml에 정의
 - 플러그인(컴파일, 문서화, 배포)을 통해 전역적인 재사용 가능
 - 빌드에 대한 대부분의 책임을 각 플러그인에 위임
 - 플러그인들은 Maven저장소(Repository)에 저장
-
- <https://mvnrepository.com/>



- root에 존재하는 xml파일
- 프로젝트마다 1개씩 운영
- 필요한 라이브러리를 pom.xml에 정의
- 해당 라이브러리 실행, 설치에 필요한 다른 라이브러리까지 관리하고 네트워크를 통해서 자동으로 다운받음

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>myproject</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <!-- Inherit defaults from Spring Boot -->
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.1.13.RELEASE</version>
    </parent>

    <!-- Add typical dependencies for a web application -->
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
    </dependencies>
```




- Maven을 대체할 수 있는 프로젝트 구성 관리 및 범용 빌드 툴
- Ant Builder와 Groovy Script를 기반으로 구축
- 자바 진영에서는 Maven과 더불어 많이 쓰이는 빌드 툴
- XML 기반의 Maven보다 동적인 빌드를 유연하게 표현 가능
- XML 기반의 Maven보다 가독성이 높음
 - xml은 구조적인 장점이 있지만 문서의 양 자체가 비대해짐
 - Gradle은 코딩에 의한 간결한 정의 가능
- Gradle 설치 없이 Gradle Wrapper를 이용하여 빌드 지원
- Maven repository를 완전 지원
- 멀티 프로젝트 빌드를 지원하기 위해 설계된 빌드 관리 도구(관리자용 서버, 사용자용 서버)
- 빌드 캐시로 빌드 속도가 빠름

- 설정 파일 : build.gradle
 - 빌드에 대한 모든 기능을 정의하는 파일
 - 빌드 및 프로젝트의 관리 환경 구성
 - 빌드방법, 라이브러리 정보 등을 기술



- implementation : 컴파일과 런타임시 필요한 의존성, 예) 일반적인 경우
- compileOnly : 컴파일에만 사용되는 의존성 정의, 예) lombok
- runtimeOnly : 런타임에만 사용되는 의존성 정의, mysql-connector, h2 database
- testImplementation : junit 관련 의존성 정의
- annotationProcessor : annotation처리관련 의존성 정의, 예) Lombok



- task는 어플리케이션 빌드부터 테스트까지 다양한 작업을 수행하는 작업 단위를 뜻
- 실행 시 콘솔상에서 gradle [task명]으로 실행
- Gradle 내부에 미리 만들어져 있는 내장의 task들과 build.gradle 파일에 사용자가 정의해 사용하는 사용자 정의 task 두 종류가 존재

```
task hello {  
    println("이것은 hello task")           //설정 단계  
    doLast {                               //실행 단계  
        println("이것은 hello task 실행한 것입니다.")  
    }  
}  
gradle hello
```

- Plugin : task의 집합
- java 프로젝트를 빌드하려면 기본적으로 java 플러그인이 필요
- id 'java' 선언만으로 java 플러그인을 통해 컴파일, 테스트, jar 파일 생성 등의 task 수행 가능

```
plugins {  
    id 'java'  
}
```



- Gradle Wrapper는 명시된 버전의 gradle이 없을 경우 자동으로 다운로드 및 설치하여 빌드 해 주는 스크립트
- 복잡한 프로젝트 설정 없이 새로운 개발자나 빌드 서버가 빠르게 작업 시작이 가능
- 특정 버전의 Gradle을 프로젝트와 함께 사용할 수 있어, 모든 개발자와 빌드 서버가 동일한 버전의 빌드 도구를 사용하는것을 보장
- 필요한 경우 쉽게 Gradle 버전을 업그레이드 할 수 있음
- 환경에 종속되지 않고 프로젝트를 빌드할 수 있음
 - gradle build를 사용하면 로컬에 설치된 gradle과 java를 기준으로 build
 - gradlew build를 실행하면 build.gradle 파일에 정의한 내용을 기준으로 build
 - 로컬 환경과 관계없이 프로젝트를 빌드할 수 있음
- Spring Initilizr 혹은 IntelliJ에서 gradle 프로젝트로 시작하면 wrapper 관련 코드가 생성됨



```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '3.2.5' // bootJar, 의존성 포함 실행 가능한 jar  
    id 'io.spring.dependency-management' version '1.1.4' // spring 의존성 버전 관리 자동화, 전을 안 써도  
                                                         // Spring Boot가 맞는 버전을 자동으로 맞춰줌  
}  
  
group = 'com.example'  
version = '0.0.1-SNAPSHOT'  
  
java {  
    sourceCompatibility = '17'  
}  
  
repositories {  
    mavenCentral()  
}  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter' // 단일 라이브러리가 아니라 여러 의존성을 묶어둔 스타터(starter)  
    testImplementation 'org.springframework.boot:spring-boot-starter-test' // (spring-core, spring-context, spring-boot, spring-boot-//autoconfigure)  
                                                         // gradle dependencies 명령으로 tree구조로 확인가능  
}  
tasks.named('test') { // 테스트 실행 엔진을 JUnit Platform (JUnit 5)으로 설정  
    useJUnitPlatform()  
}
```



```
configurations {  
    compileOnly {  
        extendsFrom annotationProcessor  
    }  
}  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    compileOnly 'org.projectlombok:lombok'  
    annotationProcessor 'org.projectlombok:lombok'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}
```

// compileOnly 설정이 annotationProcessor
// 에 선언된 의존성을 상복받음

// Tomcat, Spring MVC, JSON 처리



```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    compileOnly 'org.projectlombok:lombok'  
    annotationProcessor 'org.projectlombok:lombok'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}
```



```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'  
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    compileOnly 'org.projectlombok:lombok'  
    annotationProcessor 'org.projectlombok:lombok'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}
```




```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'  
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    compileOnly 'org.projectlombok:lombok'  
    runtimeOnly 'com.mysql:mysql-connector-j'  
    annotationProcessor 'org.projectlombok:lombok'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}
```

Spring Boot Project : 빌드

- cd restapiapp
- gradle build
- java -jar build/libs/restapiapp-0.0.1-SNAPSHOT.jar



```
(base) D:\2024\lecture\1\cloud\10주차\restapiapp>java -jar build/libs/restapiapp-0.0.1-SNAPSHOT.jar
```

```

  ____  __
 / ___/  / /
/ /   /  / /
/ /___/  / /
\____/___/_/

:: Spring Boot ::
                (v3.2.4)

```

```
2024-05-06T19:39:51.391+09:00 INFO 24496 --- [restdemo] [           main] c.example.restdemo.RestdemoApplication : Starting RestdemoApplication v0.0.1-SNAPSHOT using Java 21.0.2 with PID 24496 (D:\2024\lecture\1\cloud\10주차\restapiapp\build\libs\restapiapp-0.0.1-SNAPSHOT.jar started by software in D:\2024\lecture\1\cloud\10주차\restapiapp)
2024-05-06T19:39:51.395+09:00 INFO 24496 --- [restdemo] [           main] c.example.restdemo.RestdemoApplication : No configuration file set, falling back to 1 default profile: "default"
2024-05-06T19:39:52.647+09:00 INFO 24496 --- [restdemo] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
```



```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '3.2.4'  
    id 'io.spring.dependency-management' version '1.1.4'  
}  
group = 'com.example'  
version = '0.0.1-SNAPSHOT'  
java {  
    sourceCompatibility = '17' }  
  
configurations {  
    compileOnly {  
        extendsFrom annotationProcessor }  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    compileOnly 'org.projectlombok:lombok'  
    annotationProcessor 'org.projectlombok:lombok'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}  
  
tasks.named('test') {  
    useJUnitPlatform()  
}
```